



Visualizing complexity

Ways to see the complexity
in python project

Maxim Danilov

Who I am:



- » Maxim Danilov
- » Decades of work in Python and Django development
- » JavaScript frameworks last 10 years.
- » Award-winning mentor in engineering

PAPA

Link to slides: github.com/danilovmy/pyconpt2025



Special thanks to my family for the support:

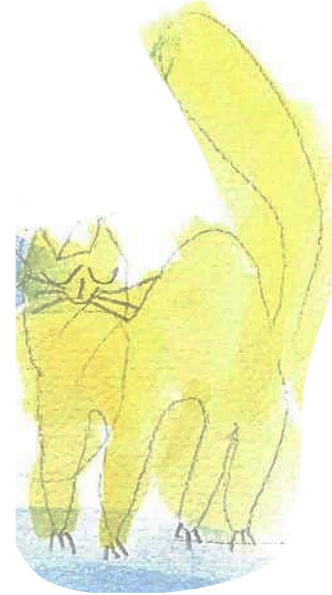


my wife
Elena



daughter
Maja

son
Mark



cat
Kisa



dog
Marselle

Thanks to my children Mark & Maja for the illustrations



7/25/2025

Maxim Danilov maxim@wpshop.at

4

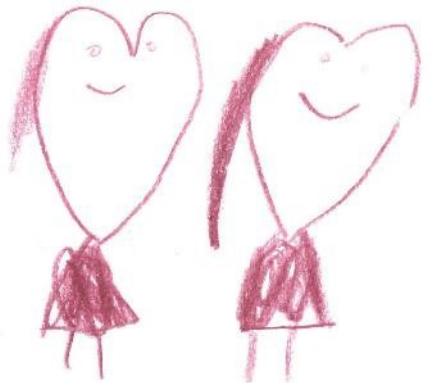


COMPLEXITY

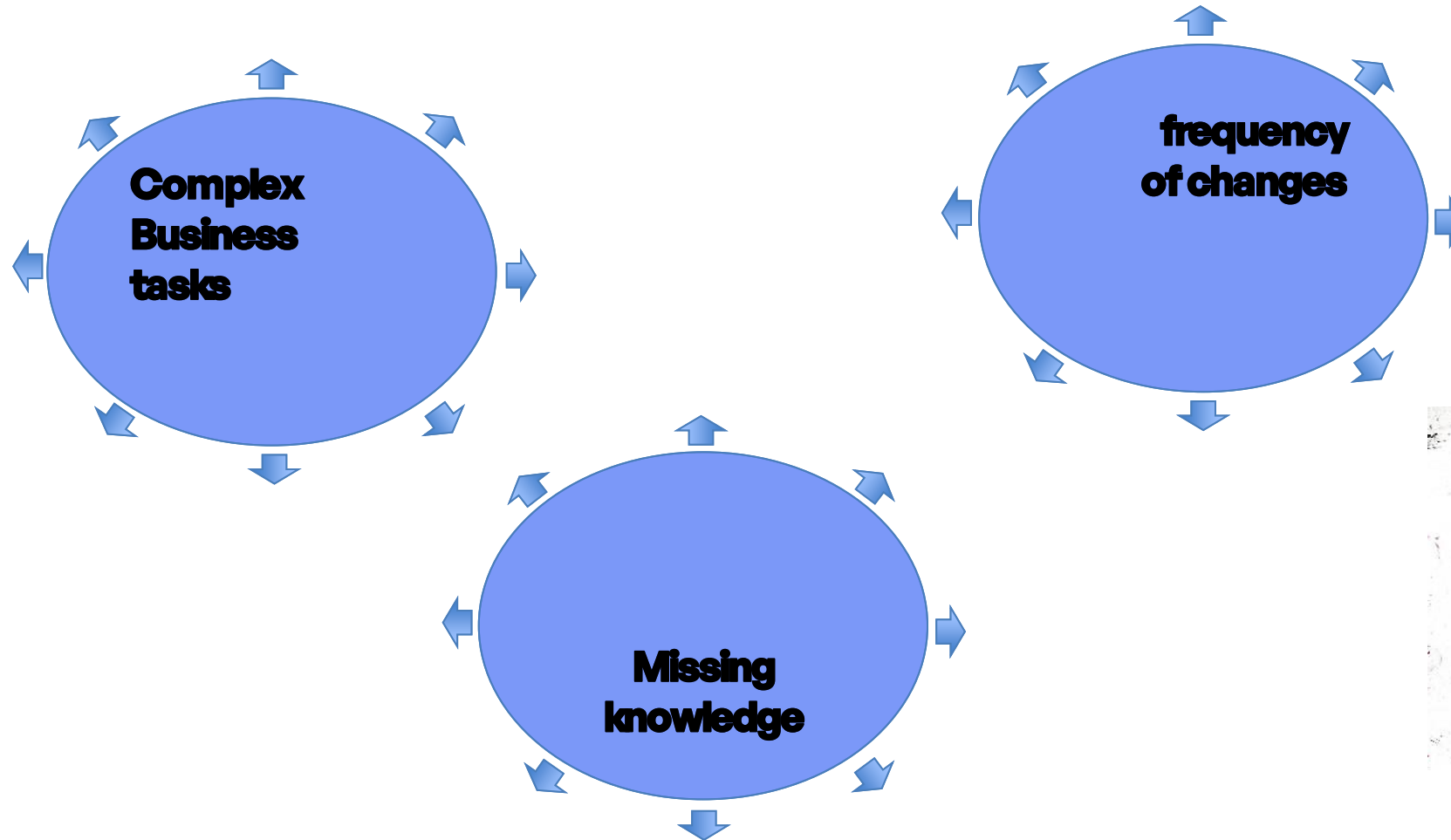
Theoretical recap

Different shadows of Complexity:

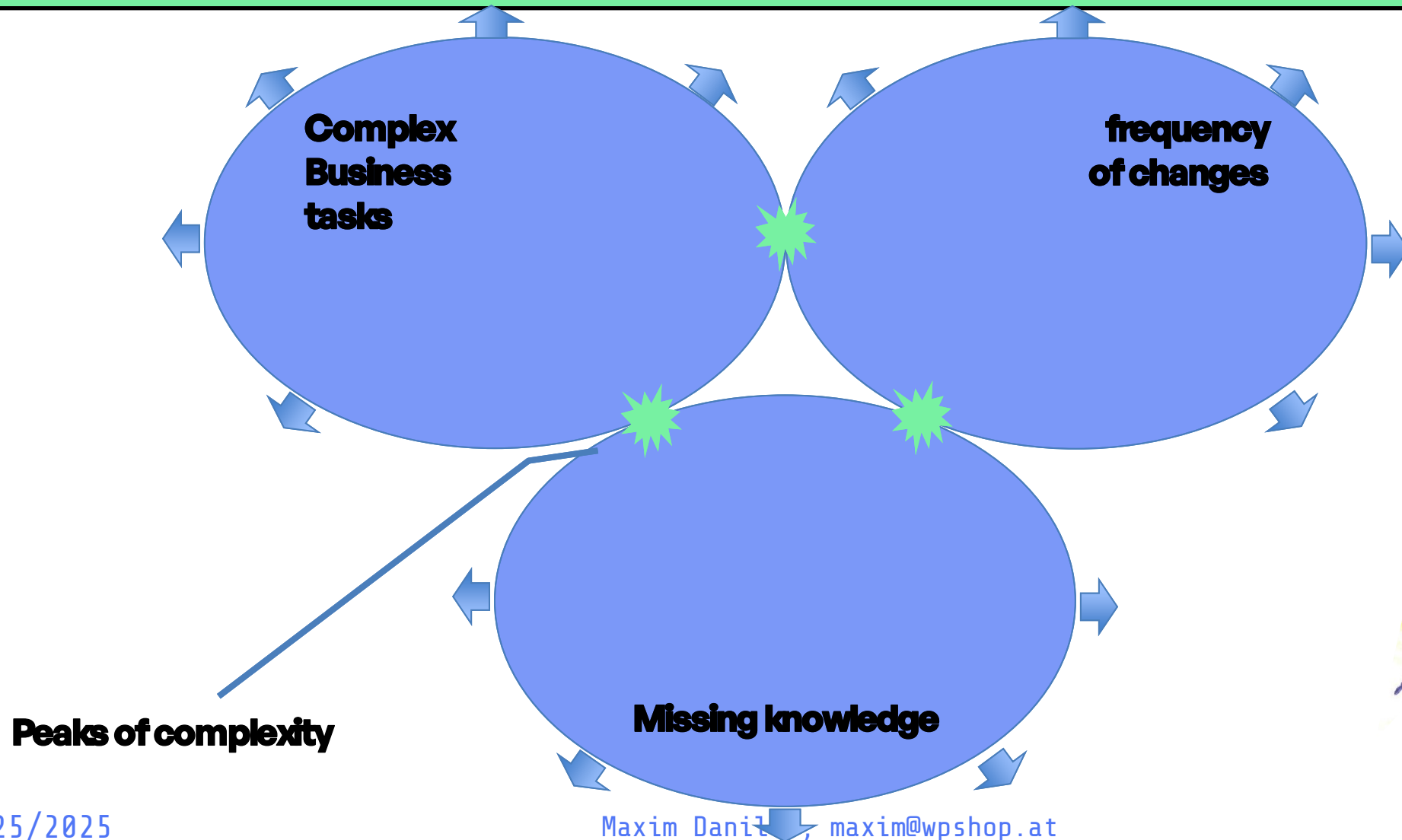
- Structural
 - Grows with relations between objects, removal by changing the coding paradigm
- Cognitive
 - Happens in development as is, reducing by following a strict coding style guide
- Accidental
 - Happens in development due compromises, removal through refactoring
- Technical (*Essential complexity*)
 - Related to algorithms. Can be frozen, reduced but can not be avoided



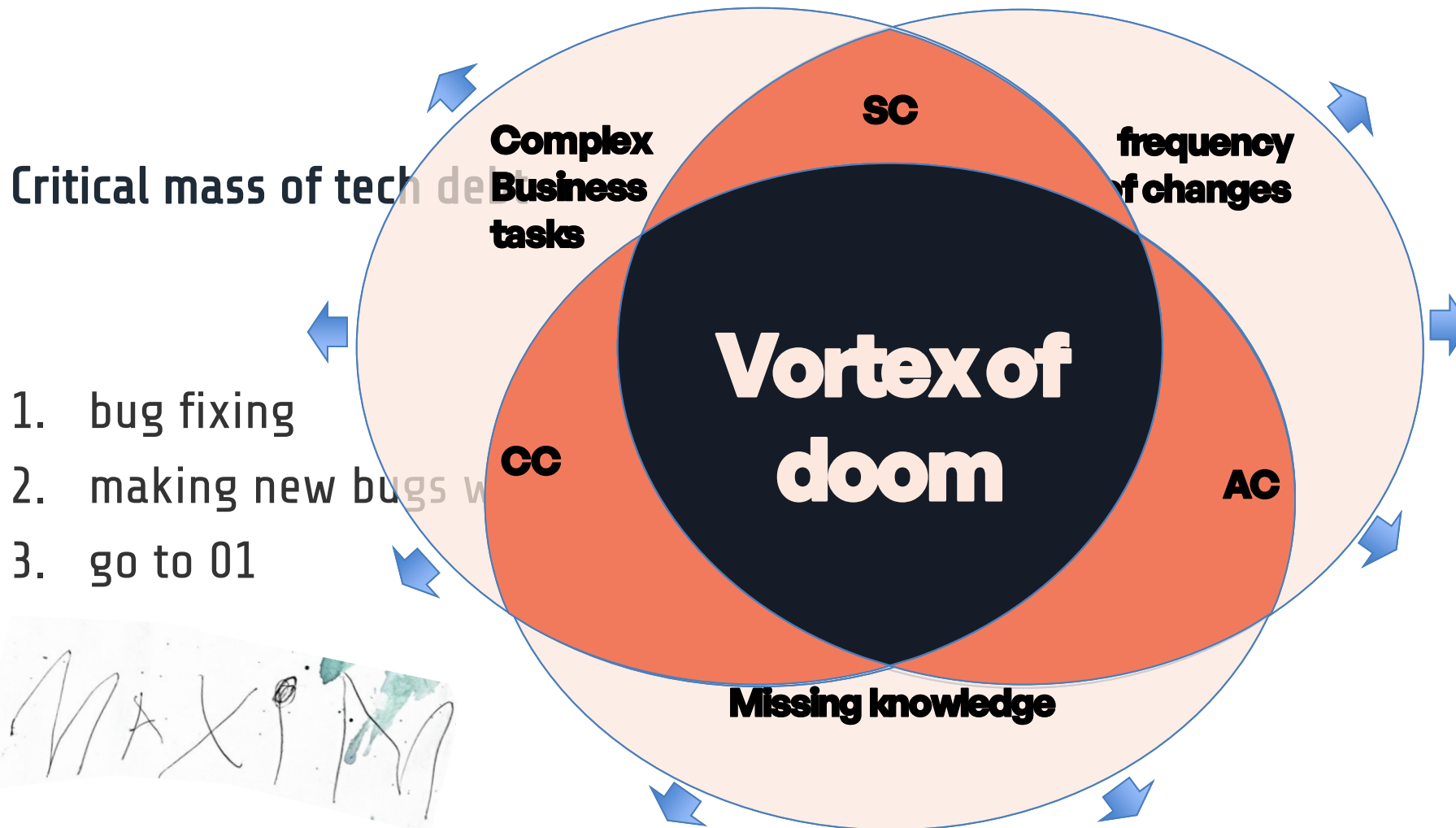
Complexity changes in project life



Matured project



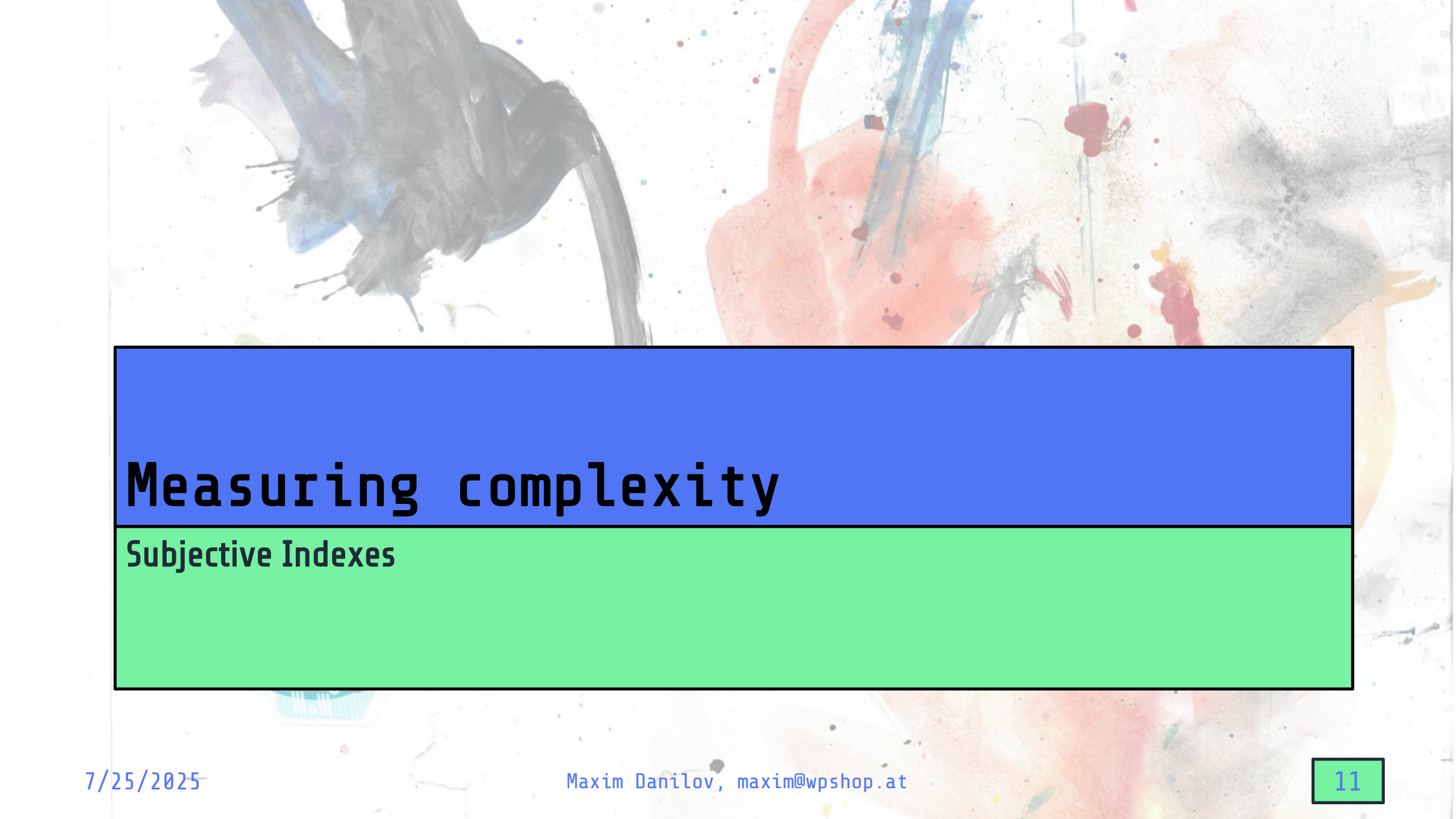
Vortex of doom



Complexity theory Summary:

- Complexity invented by humans and subjective.
- Complexity is not the same as a tech debt!
- Complexity is not a bug, but can lead to bugs.
- There is no way to keep complexity at a low level for a long time.
- Managed complexity leads to better design
- Simplicity should be set as KPI





Measuring complexity

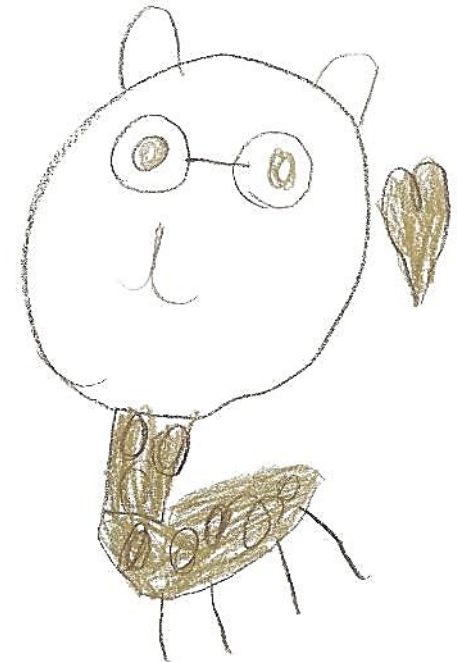
Subjective Indexes

CCL index

- CCL. Count of Code Lines.
- Measuring of project size/globality. CC
- Measuring tool: dependency-extractor, perform_extraction -> output.

Cons:

- Broken by formatters like Black, Ruff, Prettier.
- Different statements in one line
- Verbose $\neq$ Complex. Long code can be clearer

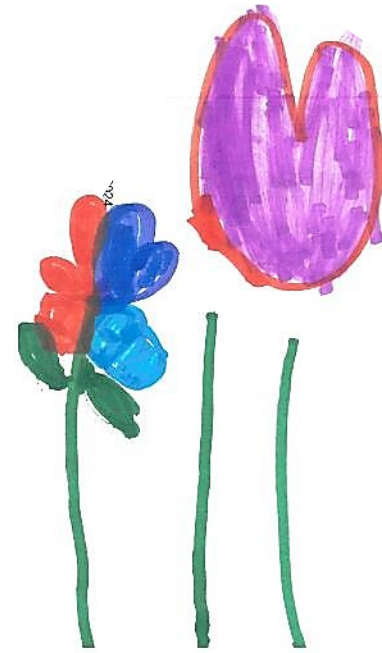


class/function index

- OOP vs FB style measurement.
 - + God Functions / God Classes
- Subjective measurement of architecture maturity. CC / SC
- Measured: dependency-extractor, perform_extraction -> output.

Cons:

- Broken by functions composition (Fastapi)
- Pytest/Unittests should not be counted.



Dependency Index

- Coupling/Cohesion index.
- Measuring of structure complexity. SC
- Measured: dependency-extractor, perform_extraction -> output.
- Cons
 - not works with dynamic imports, imports on-fly



Coverage

Coverage report: 99%

Files

Functions

Classes

coverage.py v7.9.2, created at 2025-07-25 09:55 +0100

File ▲	statements	missing	excluded	coverage
gptests.py	184	1	0	99%
plain.py	83	0	1	100%
Total	267	1	1	100%

coverage.py v7.9.2, created at 2025-07-25 09:55 +0100

Documentation Coverage (CLI)

- Doc
- Needed: 4; Found: 0; Missing: 4; Coverage: 0.0%
- Overall statistics for 153 files (15 files are empty):
- Needed: 779 - Found: 3 - Missing: 776
- Total coverage: 0.4%
- - Grade: Do you even docstring?

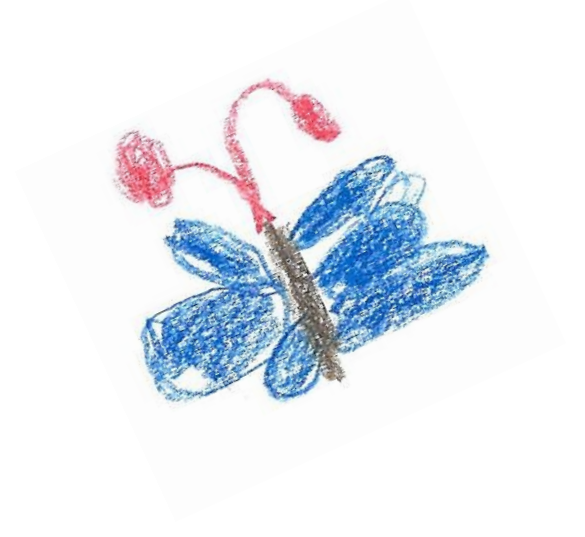
Measuring Complexity

- Cognitive Complexity:
 - Sonar Cloud, Cognitive Complexity
- Cyclomatic Complexity:
 - Radon
- Tech Debt metrics:
 - CodeClimate
- Composite code quality index:
 - Codacy



Complexity measurements Summary

- All metrics are subjective!
- Before to do anything – collect your metrics
- After some complexity refinements – collect your metrics
- Dont forget to compare it, to "*confidently make changes in any code base*"



Visualizing complexity

I can see dead peoples...

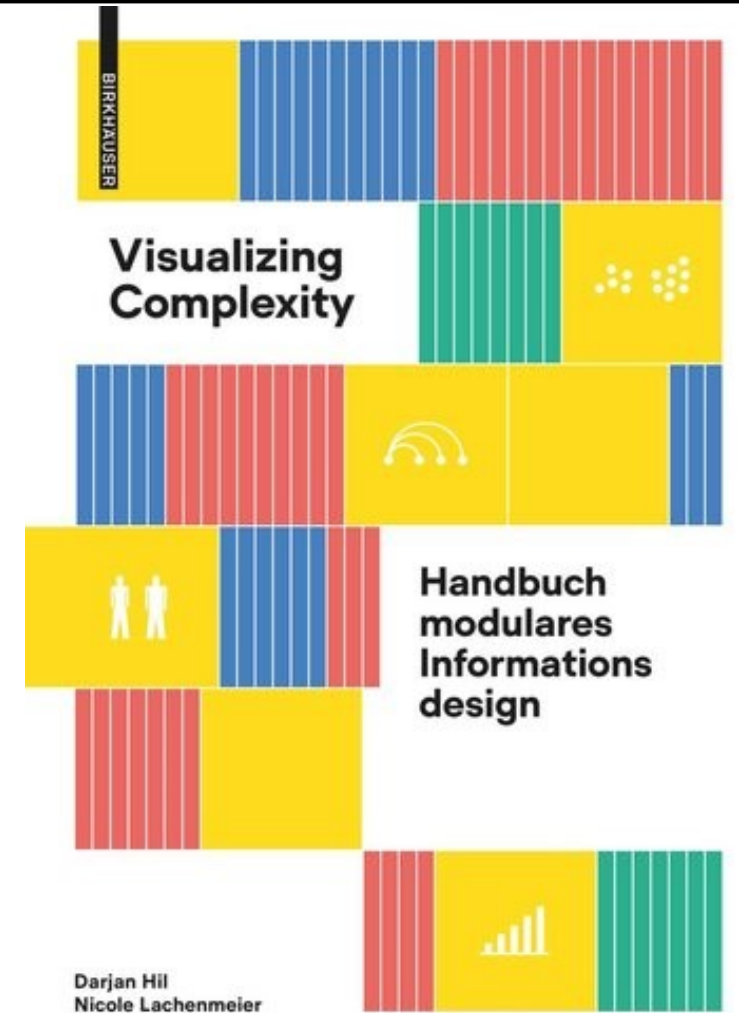
Visualization Theory

- Visualizing Complexity, Darjan Hil Nicole Lachenmeier.

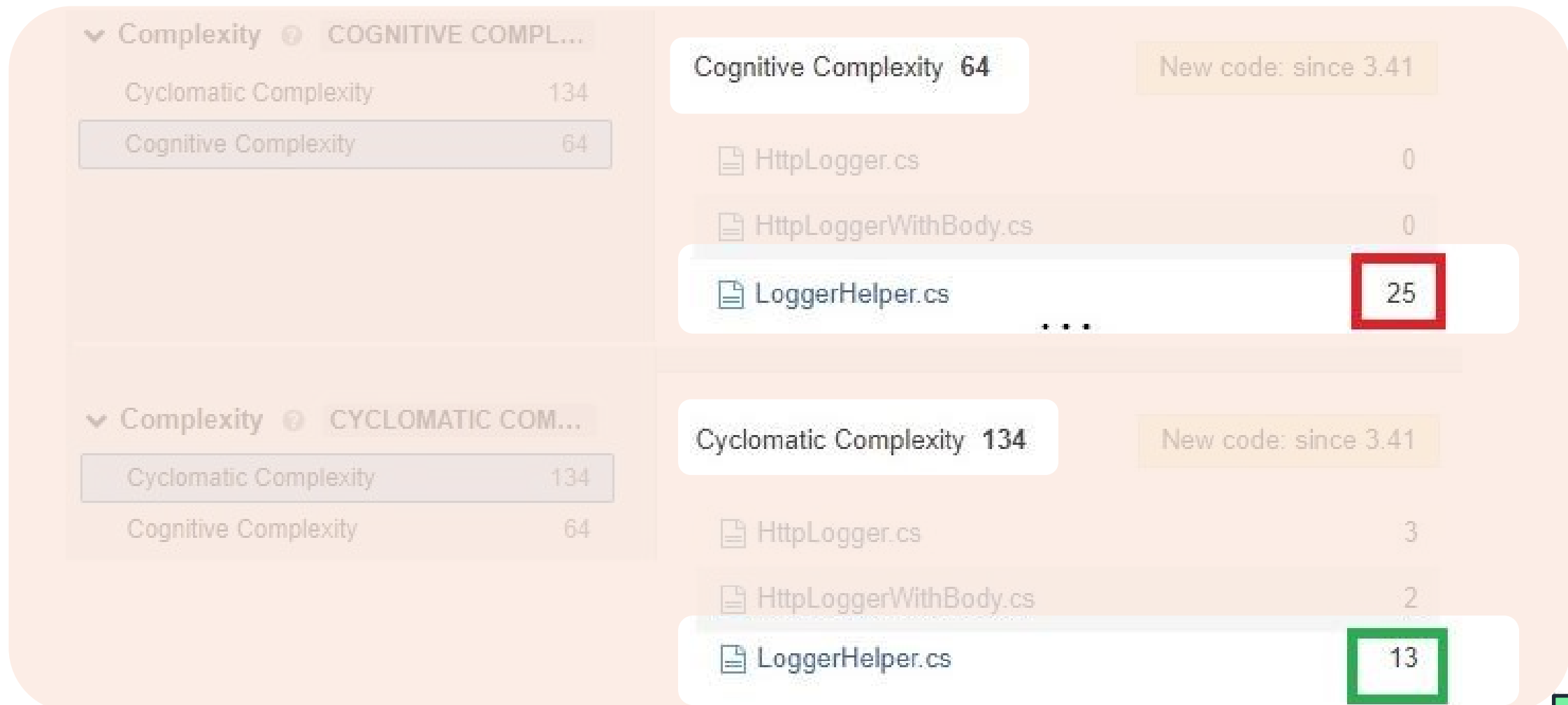
MID's

Modular Information Design System

- Data + Diagramms + Visualisierung(creativity) + Structure
- Multidimensional visualization

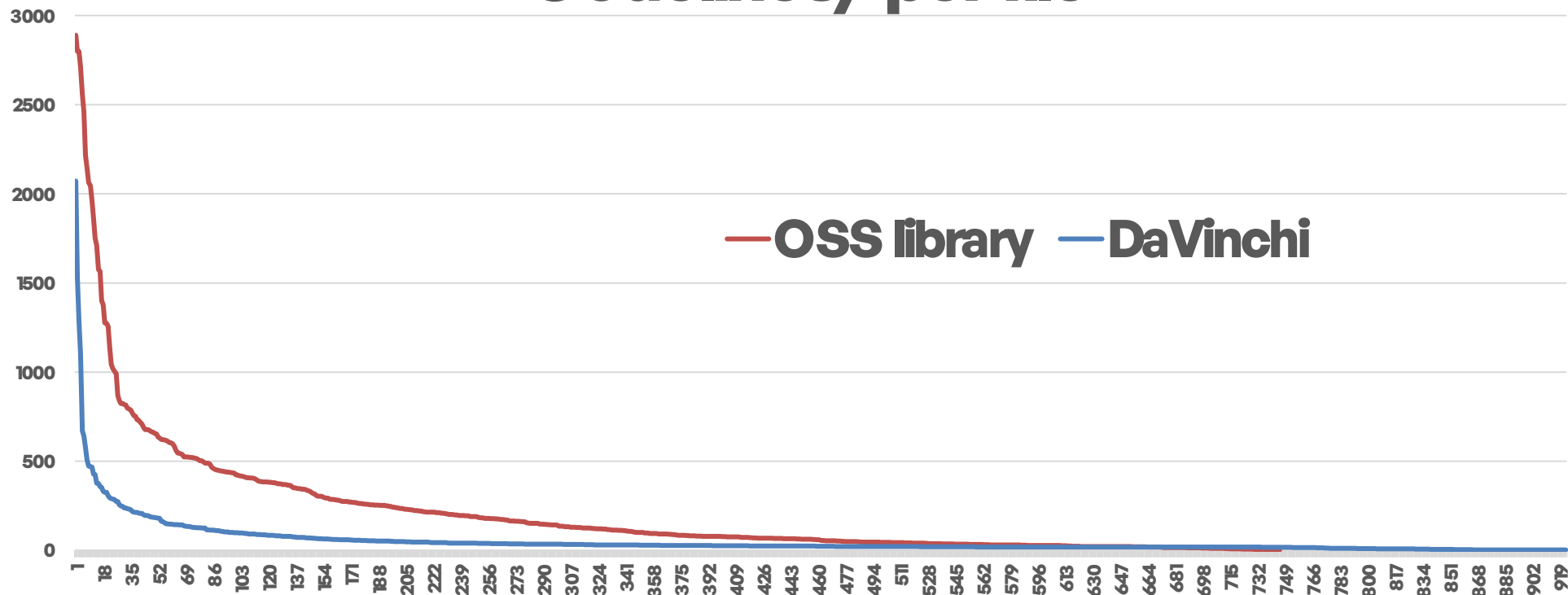


Cognitive / Cyclomatic complexity (Sonar Cloud)



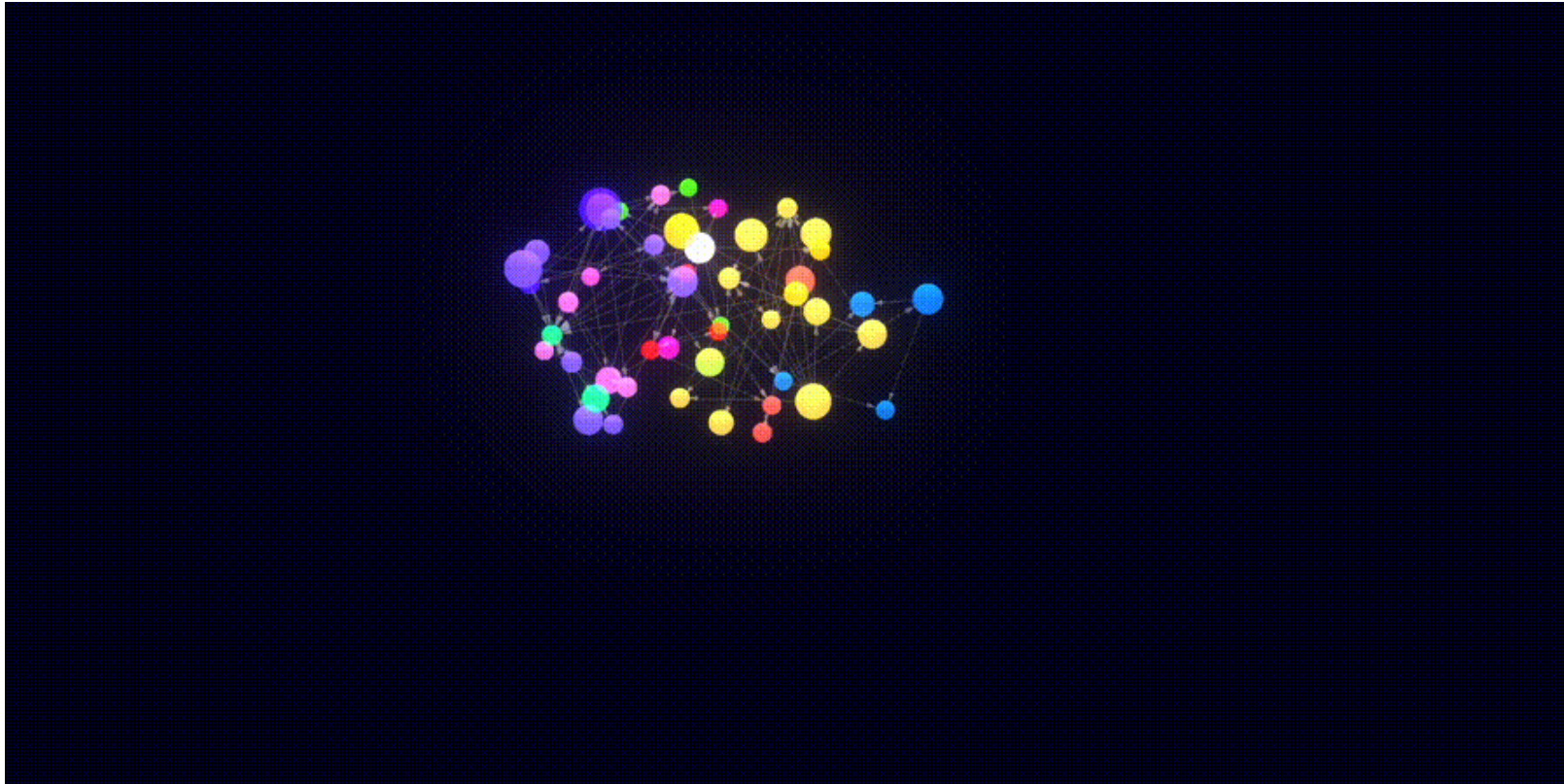
CCL example in xls diagramm (cc)

Codelines/ per file



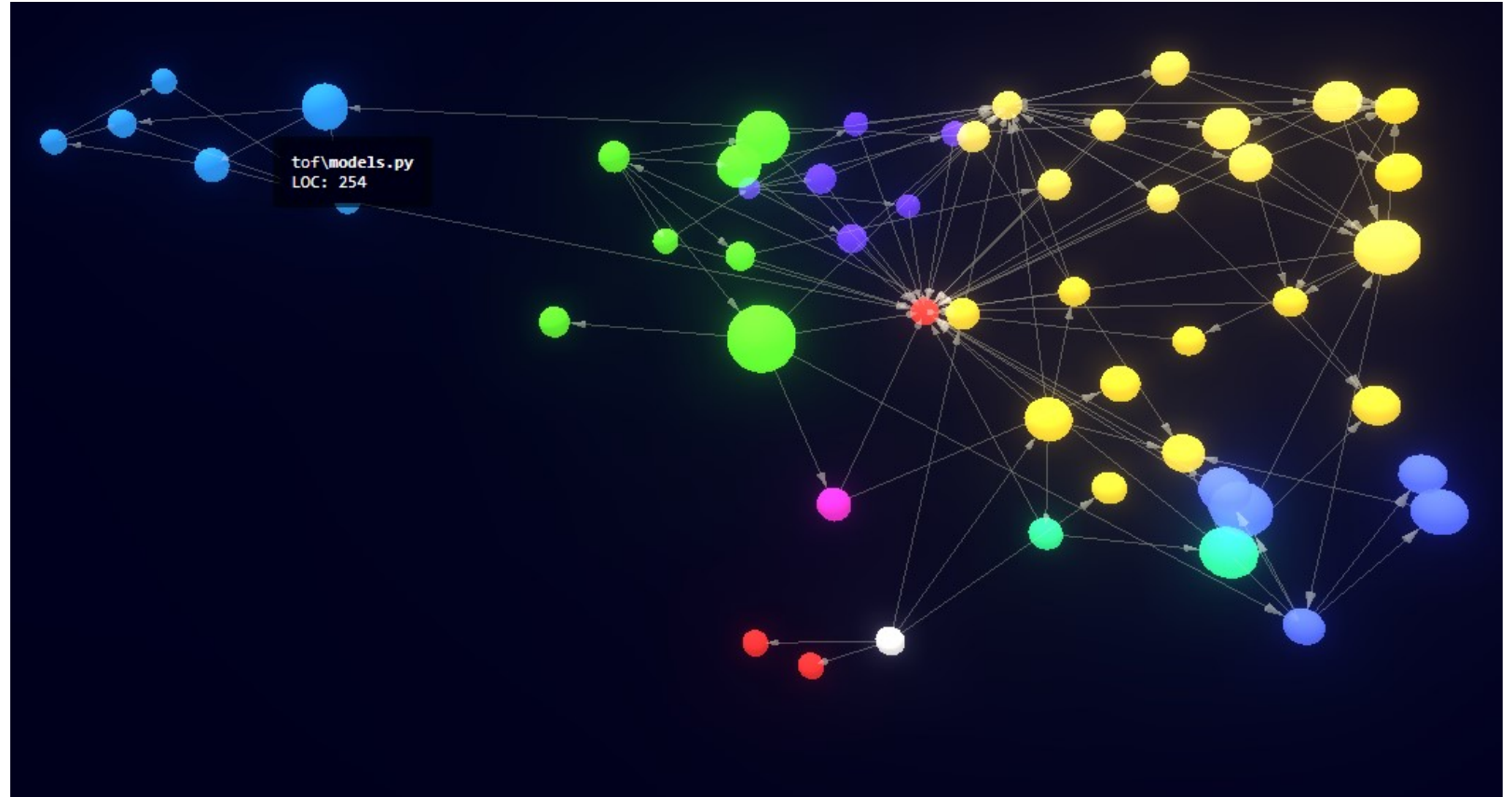
Copling / Cohegn visualization (sc)

- Visualize the **entropy** of a code base github.com/gabotechs/dep-tree



Django Project ~270000ccl

1. App-sizes
2. Internal Cohign
3. External Copling
4. Decoupling ways



Vortex of Doom visualization (sc)



NanoAPI visual maps
helping teams manage architecture



NanoAPI



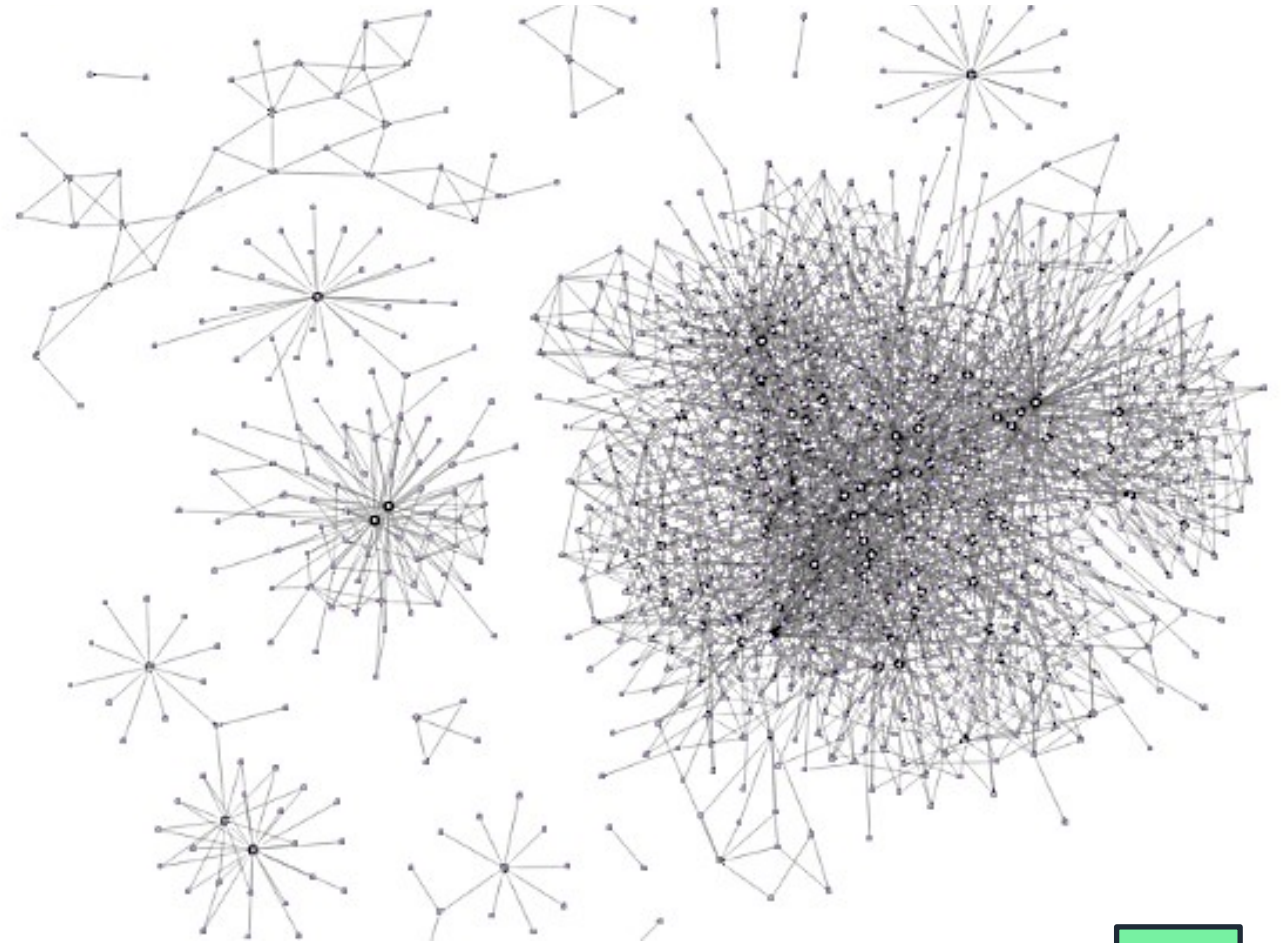
Personal



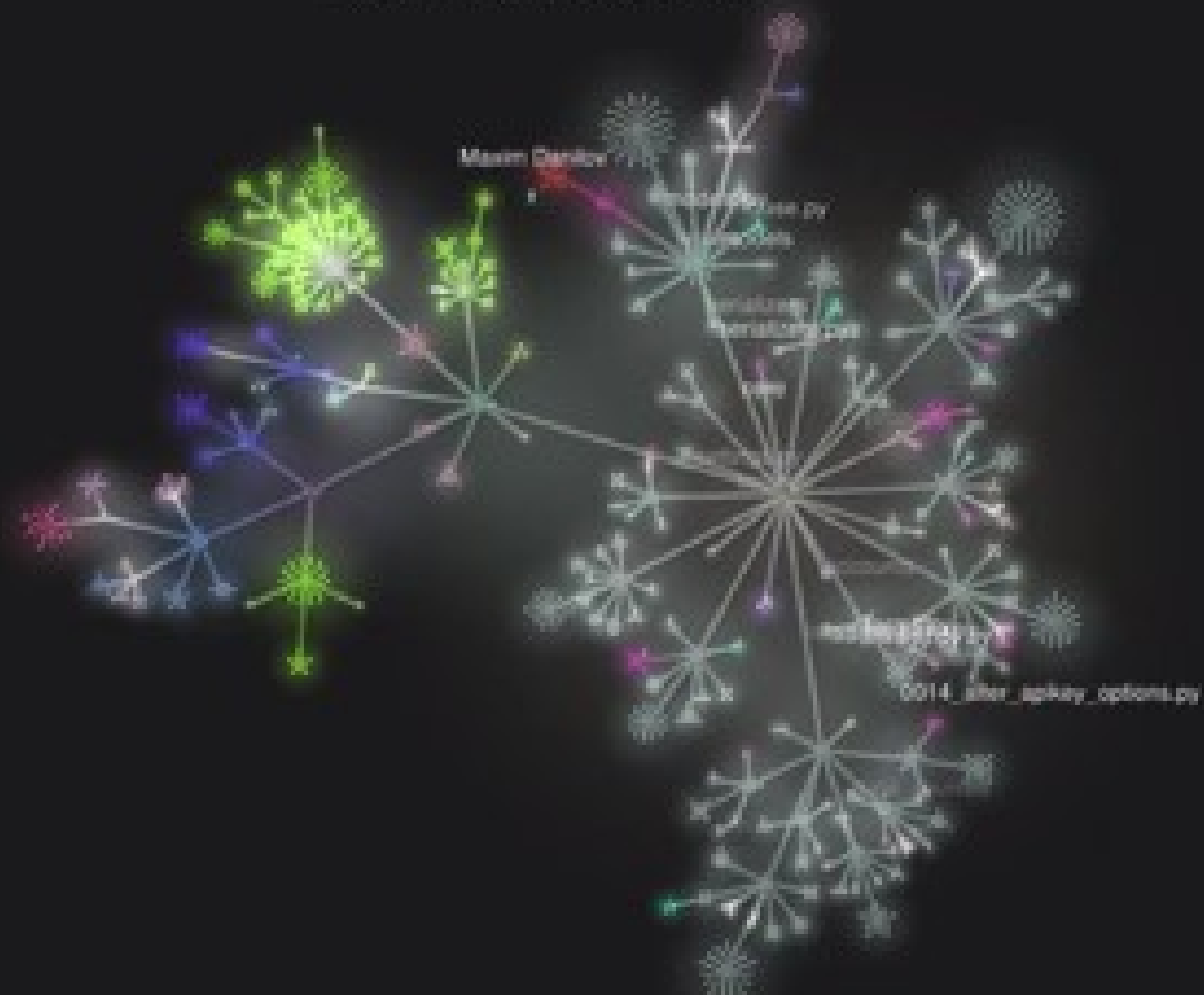
Dashboard >



Project 9 >

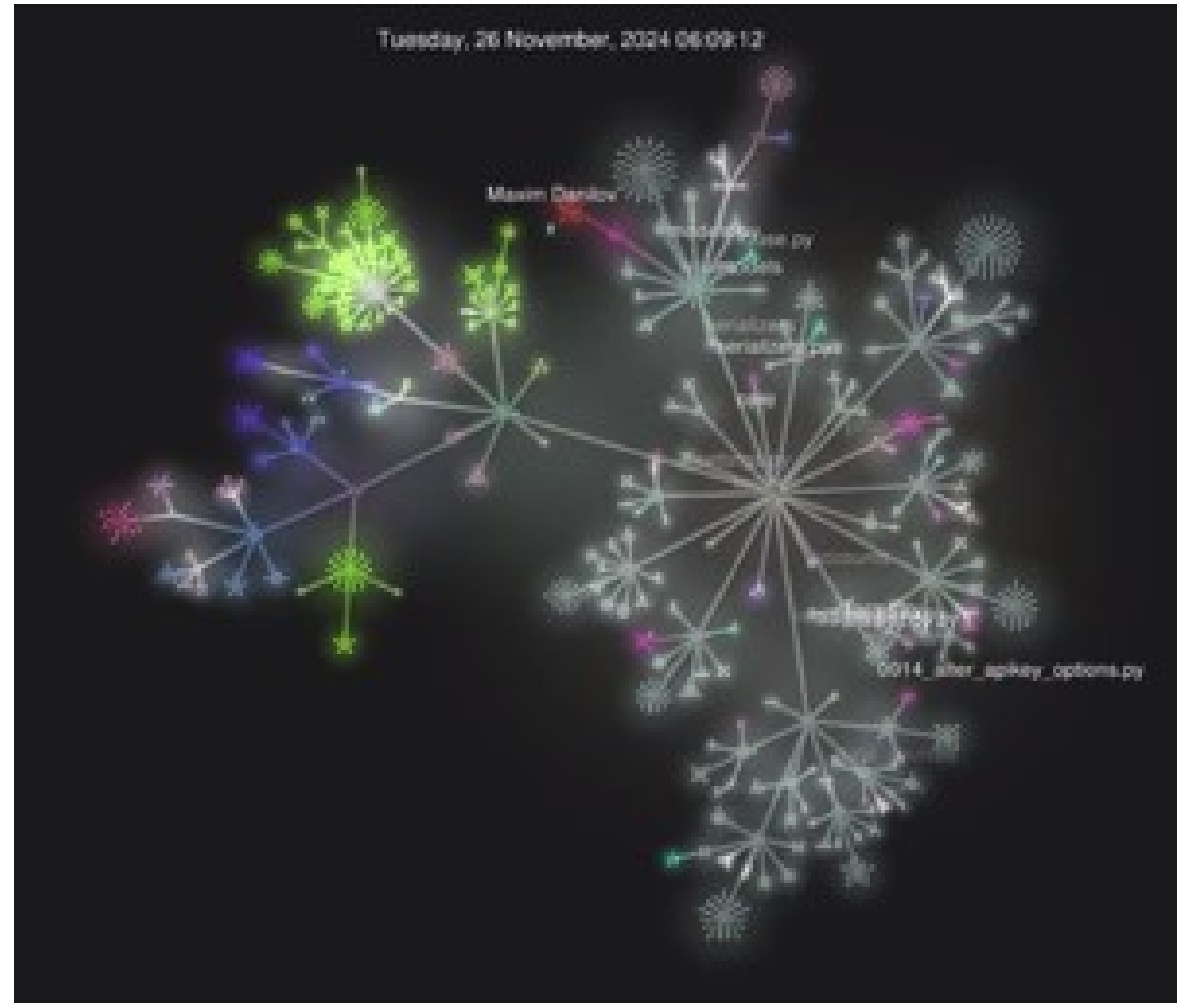


Tuesday, 26 November, 2024 06:09:12



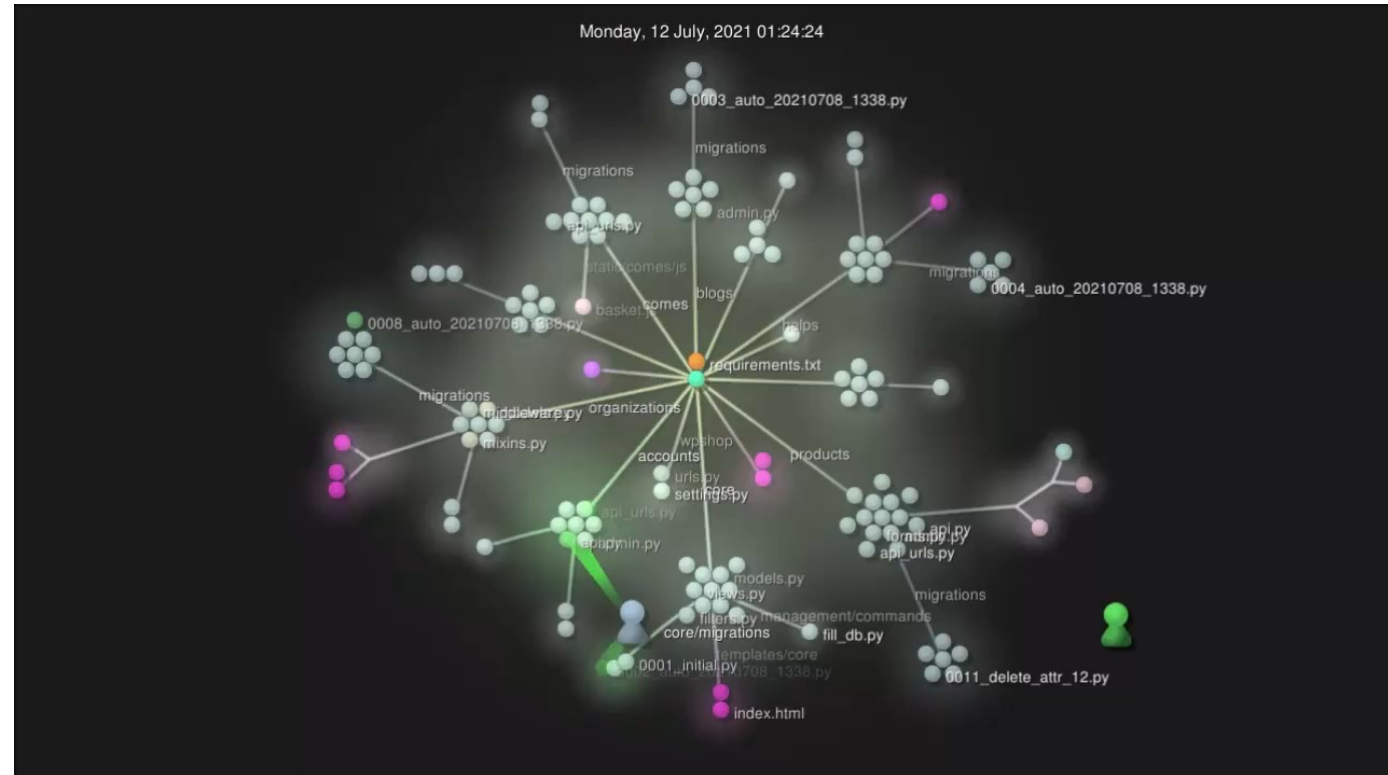
Development process visualization

1. Stability of Development
2. Matured parts
3. Modularity
4. Complexity Changing



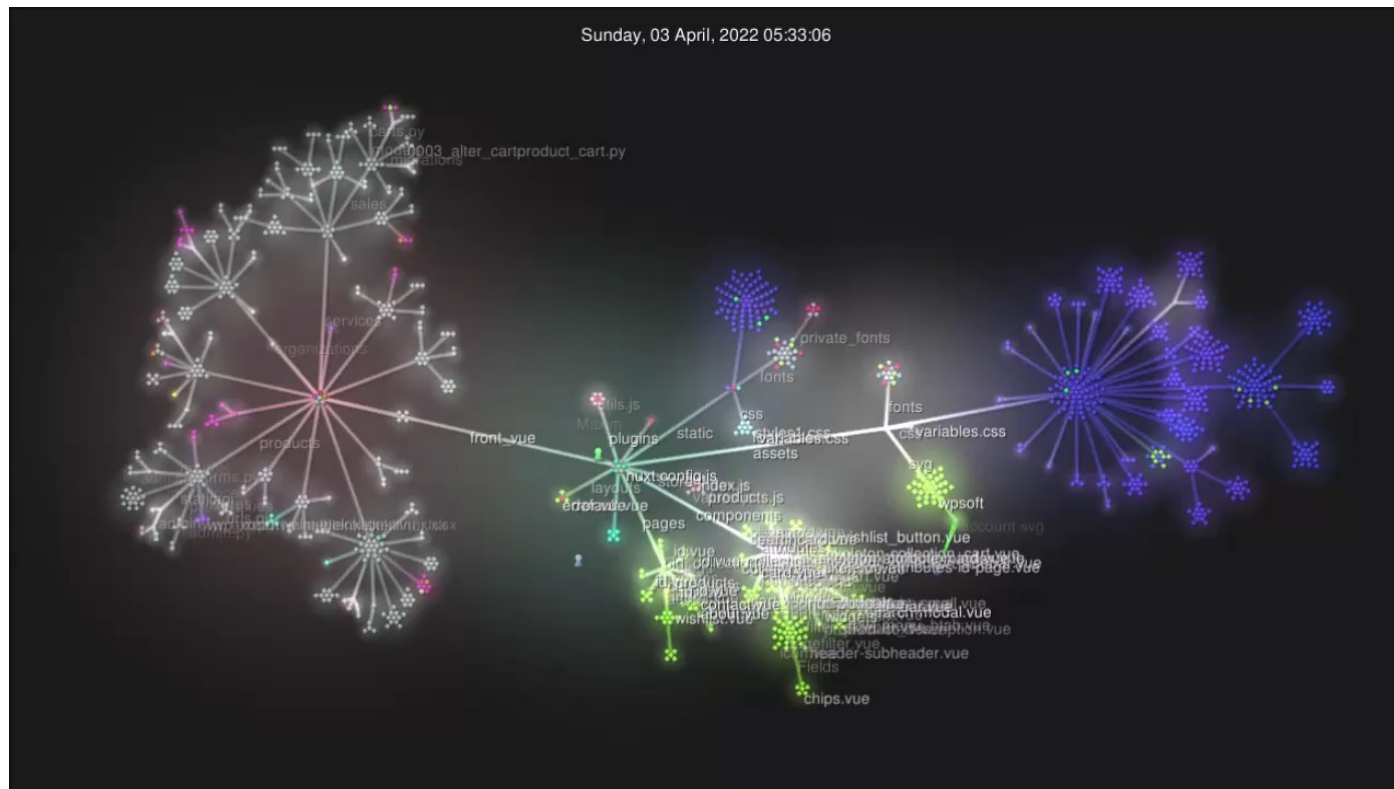
Unnoimous grow

1. Accidental complexity ↑
2. Structural complexity ↑
3. Lack of documentation ↑
4. Cognitive complexity ↑
5. Technical complexity – complex



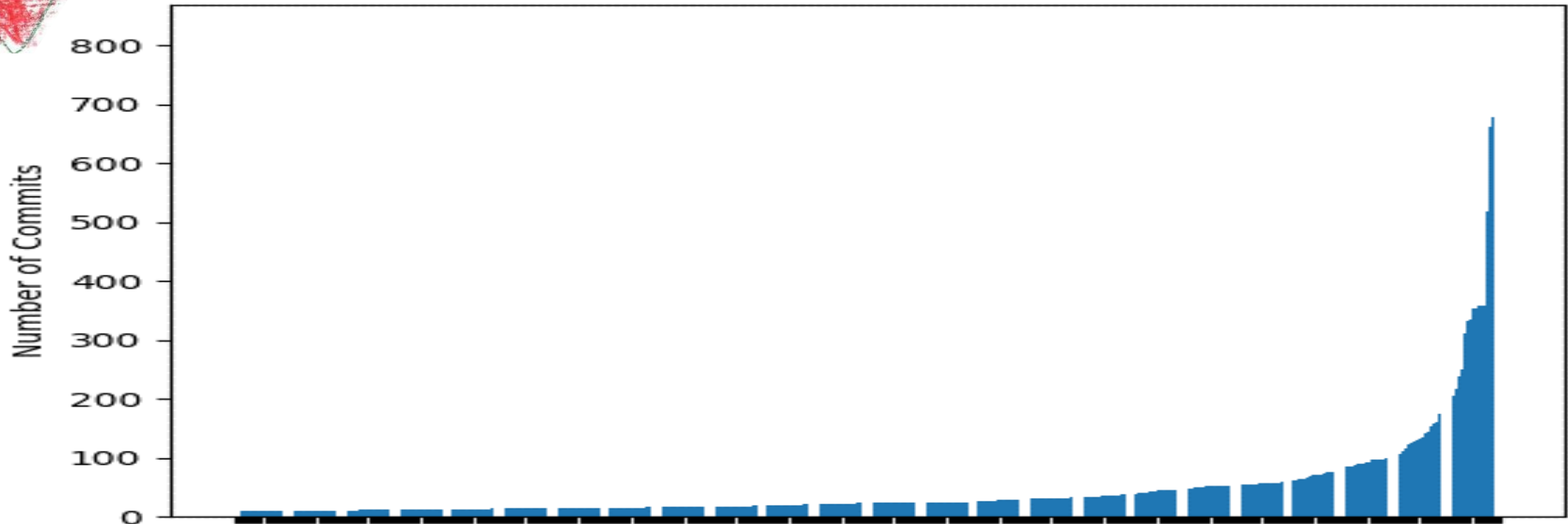
Refactoring

1. Accidental complexity ↑
2. Structural complexity ↓
3. Lack of documentation ↑
4. Cognitive complexity ↑ ↓
5. Technical complexity constant



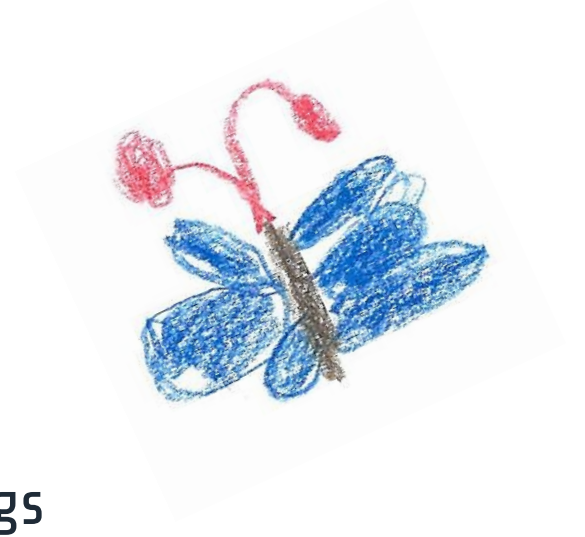
Areas of high developer activity (ac)

number of commits / per file. Highest 10% of codebase



Complexity visualizing Summary

- All what you see and interpret is subjective.
- Visualization only a trick to attract attention to problems
- Visualization is a trick to be motivated at work with complex things
- Try different tools to find a something for you





Strategies to work with complexity

Reducing complexity

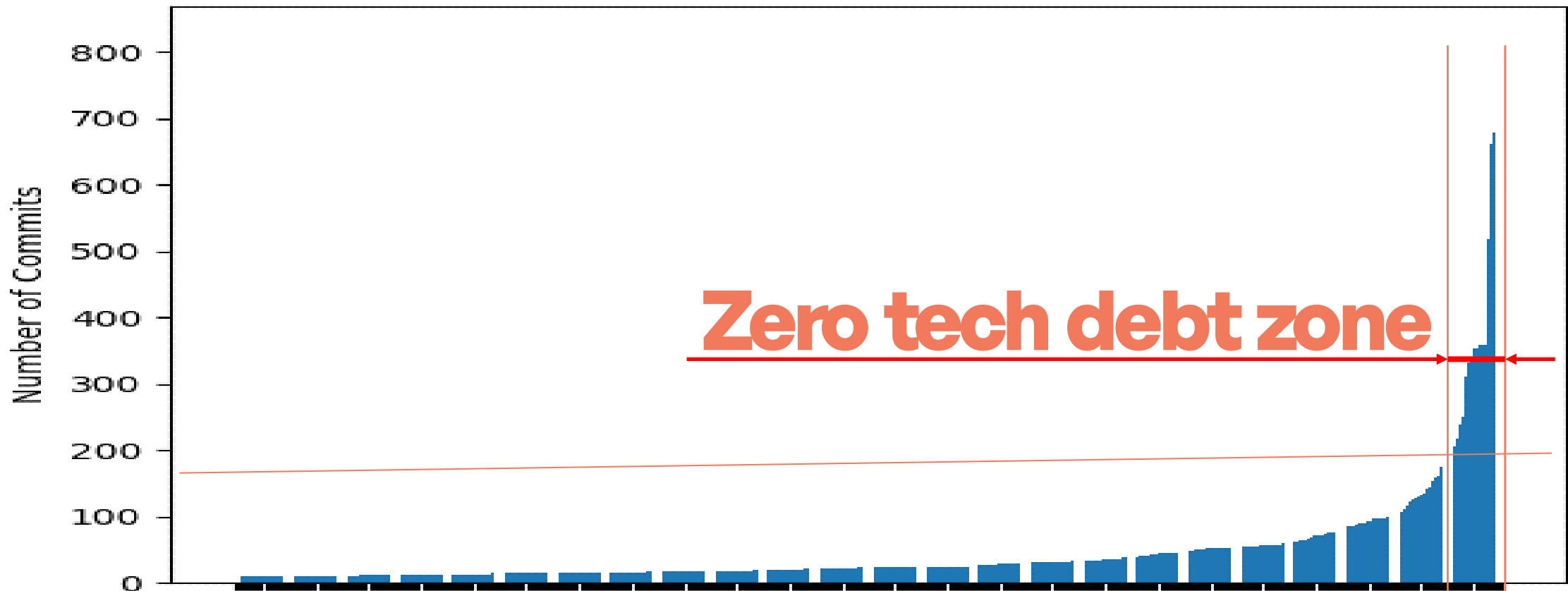
Defining refactoring Hot Zones

- Only few files change often (Michael Feathers)
- Some files must be clean from tech debt , Ignore the rest (Martin Fowler)
- 20% changes affect on 80% results (Pareto Rule)



Refactoring zone:

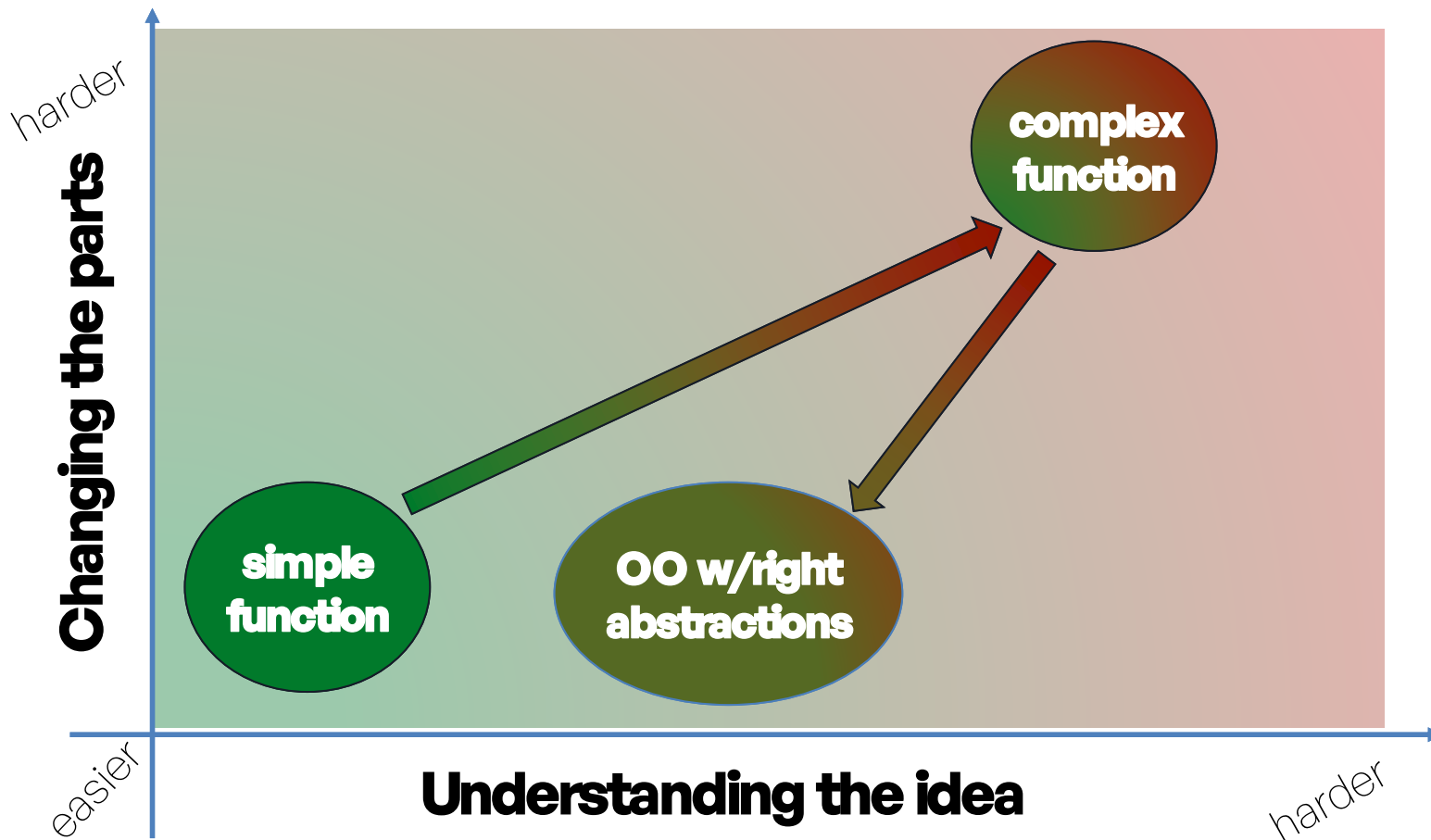
Feathers + Fowler + Pareto 20/80 Principle -> fowler_feathers_pareto.py



Changing abstraction level

OO solution is a bit costly than a simple function but far less costly than a complex function

- Sandi Metz



class/function example

DaVinci, commercial project

.py files processed: 999

summary count of classes definitions: 1709

summary count of function definitions: 2986

summary lines of code: 47828

...

35% classes / 65% functions

...

average code lines per file: 42

average quantity class/function per file: 5

average length of class/function: 10 lines

Famous OSS project

.py files processed: 885

summary count of classes definitions: 1942

summary count of function definitions: 9043

summary lines of code: 162865

...

20% classes / 80% functions

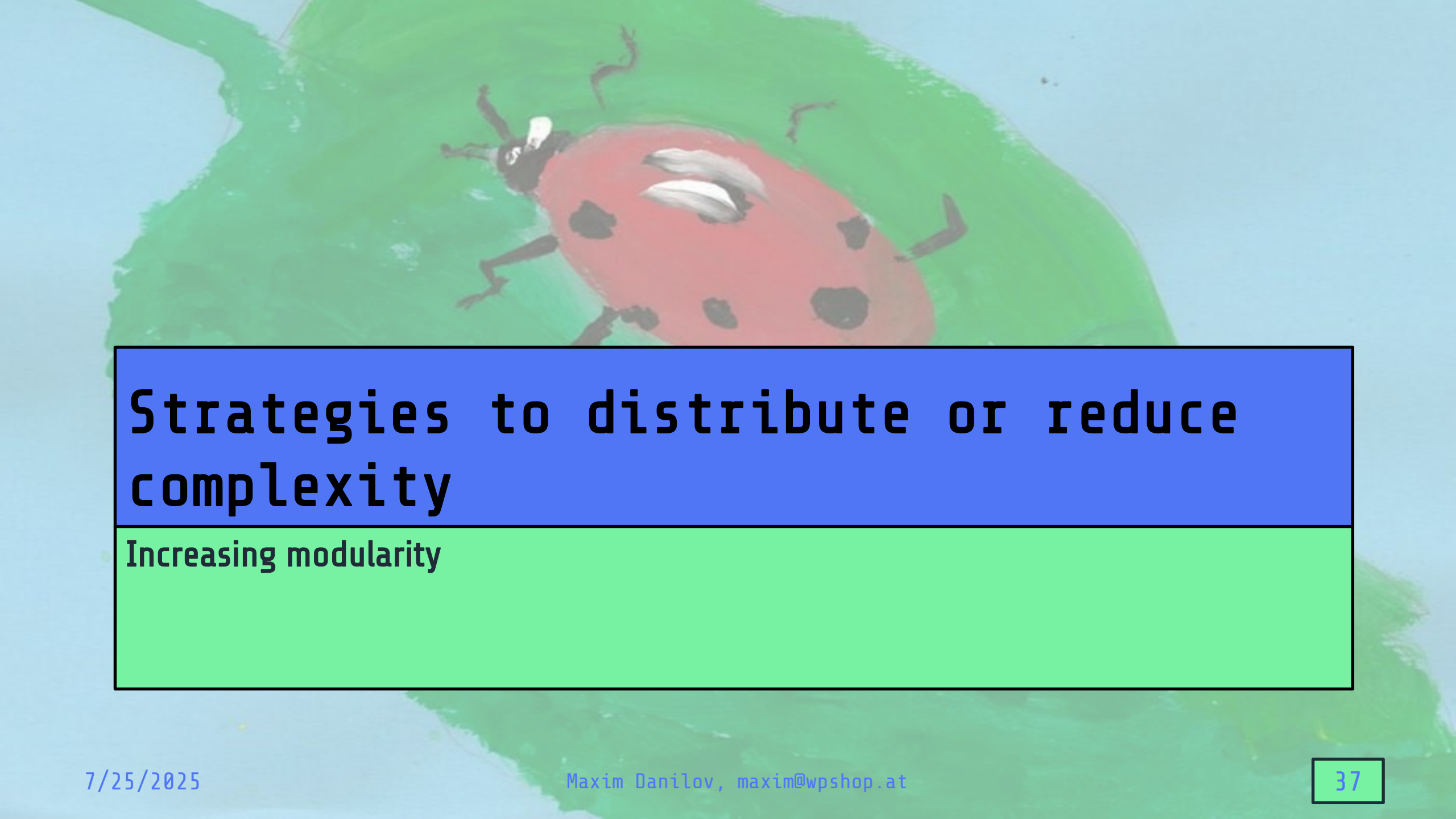
...

average code lines per file: 212

average quantity class/function per file: 15

average length of class/function: 15 lines



A painting of a ladybug on a green leaf. The ladybug is red with black spots and is positioned in the upper center of the frame. The leaf is green and occupies the left and center portions of the image. The background is a light blue sky.

Strategies to distribute or reduce complexity

Increasing modularity

Modular Monolith

1. *Organize your business logic as modular as possible*
2. *Break it up across multiple repos*
3. *Keep those components as a monolith*
4. *Get serious reduction in complexity and a simplified management process*

Kelsey Hightower

Dependency-extractor

Input:

```
>python ast_extractor.py -r project_root -f path_to_file
```

Output:

```
path\to\project\logs\ast_imports_report.txt  
path\to\project\logs\ast_classes_report.txt  
path\to\project\logs\ast_imports_per_line_report.txt
```



Example:

...

```
webhook_controller.py:12: SyntaxWarning: invalid escape sequence '\&  
find 1894 imports
```

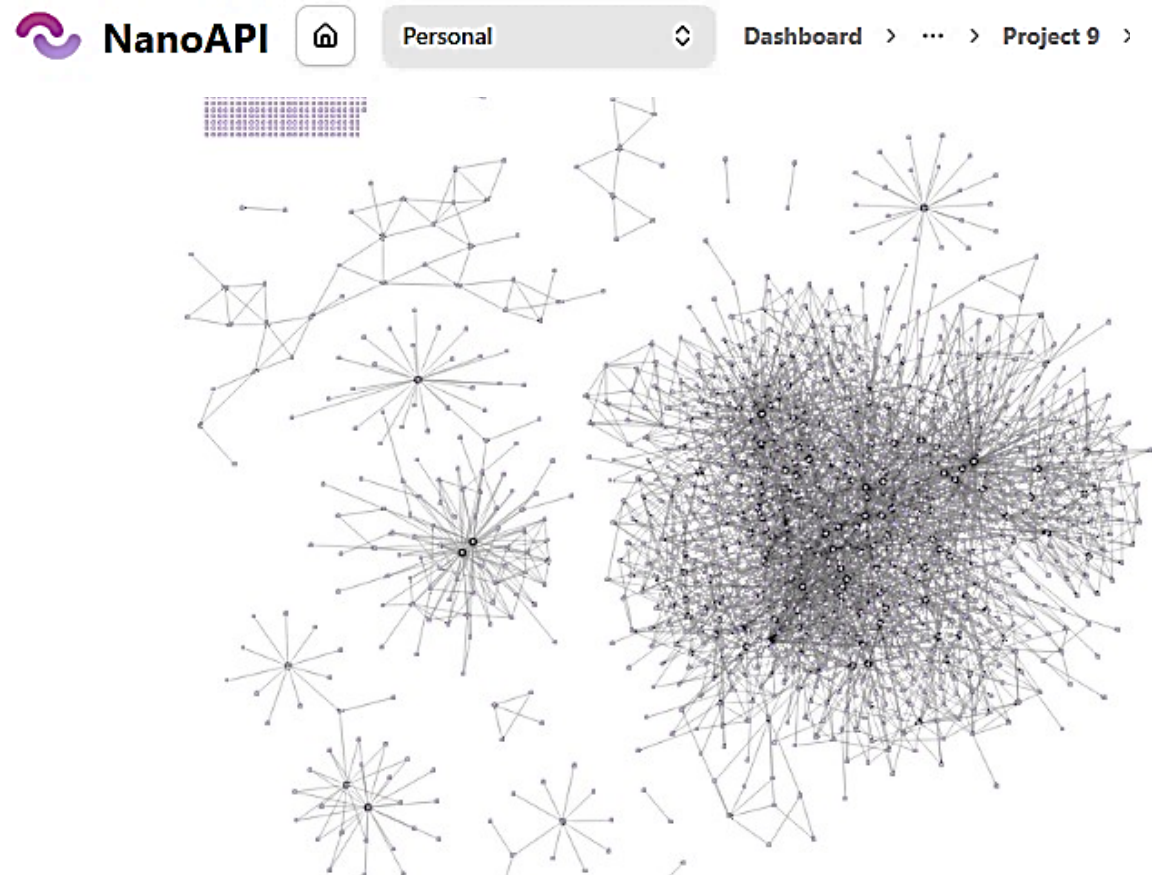
Modules extraction



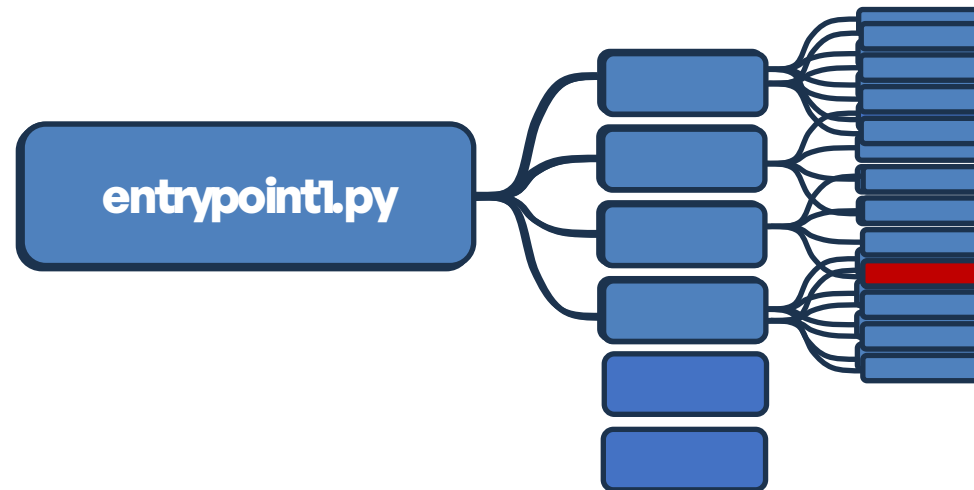
NanoAPI

helping teams manage architecture

1. *See the structure*
2. *Extract independent modules*
3. *Die with the rest*



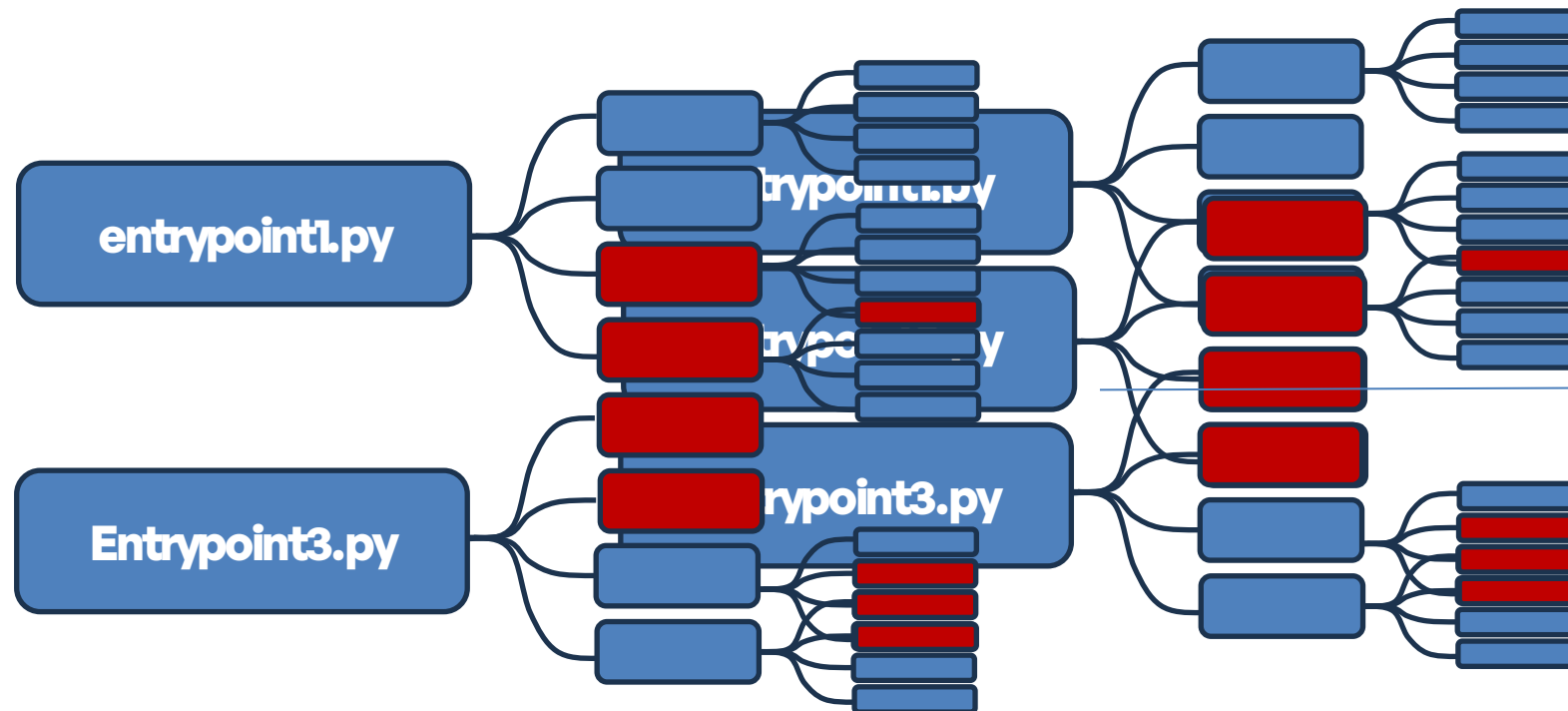
Monorepo / monolith



Modular monolith.

- Monorepo / monolith

- Modular repo / Modular monolith



Abstract level example:

DaVinci, commercial project

```
importlib detected ... 1 time  
.py files processed: 999
```

```
find 1000 dependencies  
find 24 libraries to install  
find 96 built-in dependencies
```

```
search of dependencies finished
```

Famous OSS project

```
importlib detected ... 103 times  
.py files processed: 885
```

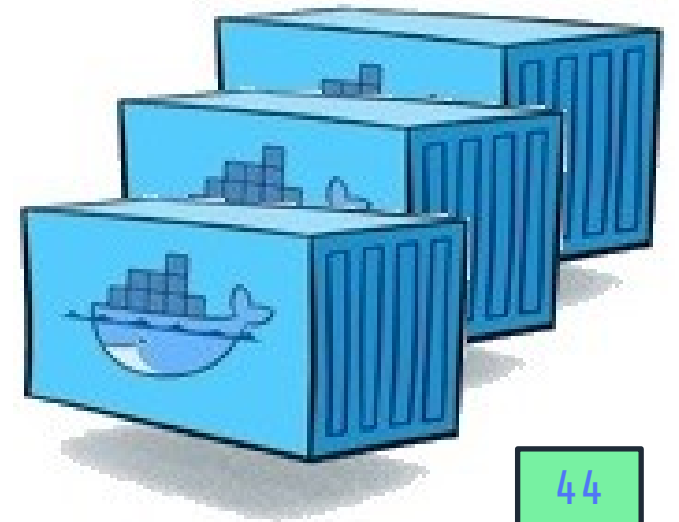
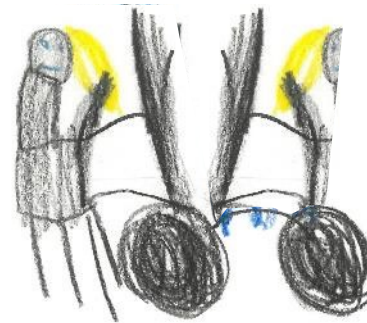
```
find 895 dependencies  
find 25 libraries to install  
find 1340 built-in dependencies
```

```
search of dependencies finished
```



Isolate without refactoring

- AST-based parsing and collecting imports node
- No runtime coupling with the rest of the codebase
- A working entry point with all internal dependencies.
 - Easily containerizable as standalone service.
 - It is possible to pack one or more entry points in one container.



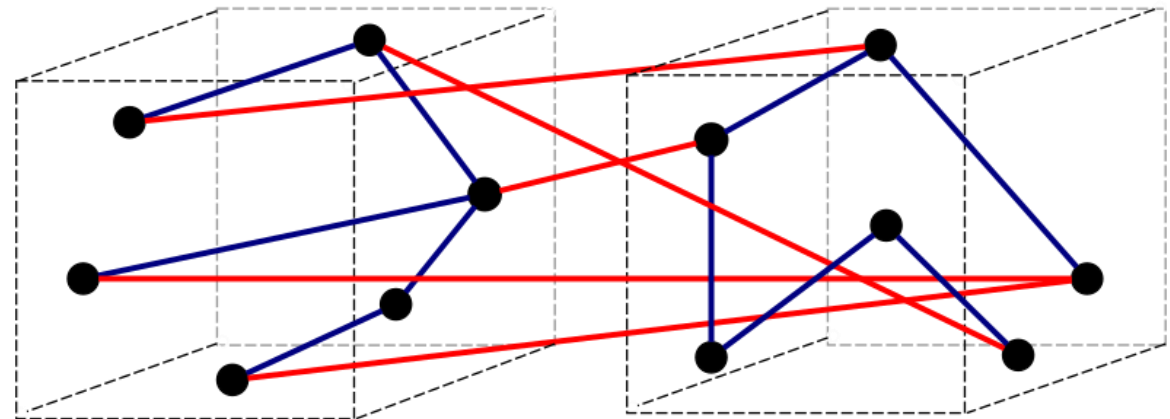
Result of extracting for one entrypoint: huge Coupling

Source:
2700 files
750 folders
21 app

- actions
- assets
- aws
- clients
- commands
- common
- consumers
- daemons
- data
- domain
- domain_areas
- entrypoints
- infrastructure_services
- mapper
- repository
- requirements
- scripting
- scripts
- server_stub
- service
- __init__.py

Result:
750 files
234 folders
11 apps

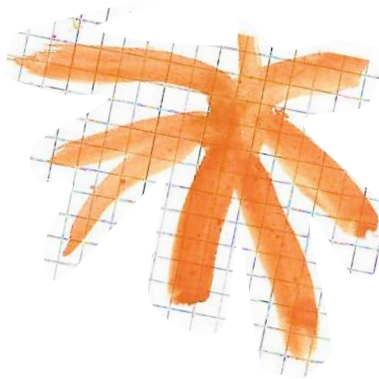
- actions
- aws
- commands
- common
- domain
- domain_areas
- entrypoints
- infrastructure_services
- mapper
- repository
- service
- __init__.py



Background for changes. Classes.

module_extractor > logs > ≡ ast_imports_report.txt

```
326 typing
218 datetime.datetime
194 dataclasses.dataclass
131 sqlalchemy.Column
112 api.domain_areas.common.utils.enums.Enum_
112 sqlalchemy.Integer
100 api.domain_areas.common.persistence.base.SqlAlchemyBase
91 sqlalchemy.String
87 logging
87 abc
84 typing.List
77 api.domain_areas.observations.domain.models.observation.Observation
70 datetime.date
65 api.domain_areas.common.domain.models.period.Period
65 sqlalchemy.ForeignKey
```



logs/ast_imports_report.txt

Increasing modularity recap:

- 0% refactoring
- Extracted module can be run standalone to Isolate bugfixing
- Extracted module can be run in monolith
- .
- Reduced container image, reduced download time, faster tests and deploys
- Extract procedure can be run exactly before creation of docker image.



More about: Kamil Grzybek, **MODULAR MONOLITH: A PRIMER**

Strategies to distribute or reduce complexity

Imports control

KIS

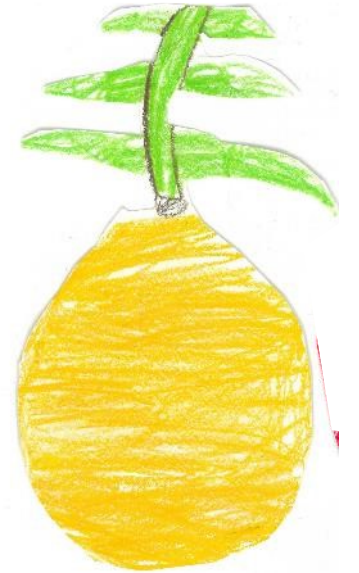
- Keep Imports Short

```
from django.db.models.fields import CharField
```

- VS

```
from django.db import models #recommended by django
```

```
from django.db.models import CharField #through __all__ in __init__.py
```



Deep nested logic

```
import api.domain_areas.observation.domain.models.observations.Observation
```

- Structural complexity ↑ ↑ ↑ ↑ ↑
- Mental load ↑ ↑ ↑ ↑ ↑
- TDD – Three dots deprecation rule in styleguide
- Linters, like **ruff** control dot-depth



Complexity hotspots

module_extractor > logs > ≡ ast_modules_report.txt

73 · legacy\api\service\service_factory.py

45 · legacy\platform_openapi_server\dto__init__.py

35 · legacy\api\repository\repository_factory.py

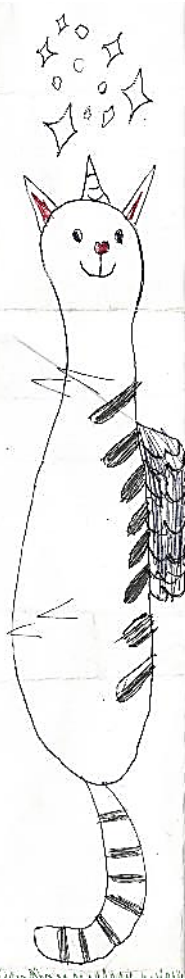
logs/ast_modules_report.txt



- Lines count ↑
- Coupling ↑ ↑
- Mental load ↑ ↑ ↑

Increasing modularity recap:

- Control imports – control coupling
- Imports with many dots required more awareness about project
- avoiding short imports from dependencies lead to fragile structure
- We can operate with 7 ± 2 concepts. Many imports increase cognitive load
- Refactoring imports usually does not require testing



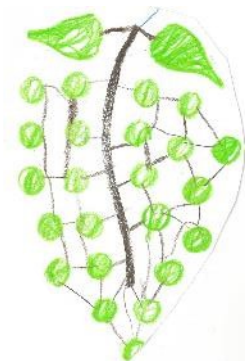
Strategies to distribute or reduce complexity

Type hints

Type Hints: Blessing or Curse?

```
from sdk import Tracker, ValidationAction
from sdk.executor import Dispatcher
from sdk.forms import FormValidationAction
from sdk.events import EventType, SlotSet

import actions.Shared.ActionServerConstants as Constants
```



Codeium: Refactor | Codeium: Explain

```
class CustomFormValidationAction(FormValidationAction, ABC):
```

```
    ....
```

Codeium: Refactor | Explain | Generate Docstring | X

```
    .... async def run(self, dispatcher: Dispatcher, tracker: Tracker, domain: Dict[Text, Any],) -> List[EventType]:
    ....     events = []
    ....     if not tracker.get_slot("slot_form_initialized"):
    ....         events.extend(await self.extra_run_logic(dispatcher, tracker, domain))
    ....     events.extend(await super().run(dispatcher, tracker, domain))
    ....     return events
```


What if



What if we replace all type hints with “typing.Any”,
then remove unused imports??

Input:

```
>python perform_extraction.py -r project_root -f path_to_file  
>autoflake --remove-all-unused-imports --inplace
```

Coupling for code with and without type hints:

A project with type hints

- find 560 dependencies
- find 21 libraries to install
- find 53 built-in dependencies
- summary files with hints: 414
- summary lines of code: 39638
- summary type hints used: 4153
- Finally package: 787 files in 234 folders, 1,46 MB.

The same project without type hints

- find 505 dependencies
- find 20 libraries to install
- find 52 built-in dependencies
- summary files with hints: 362
- summary lines of code: 29901
- summary type hints used: 4075
- Finally package: 731 files in 233 folders, 1,19 MB.



Alternatives for Type-Hints

- Type-hints in stub files (*.pyi , PEP 484)
 - + 0% refactoring
 - + generation, update, check can be automated
 - + full ide and linters support
 - + don't create noise in code file
- Type-hints in docstrings
 - + ready for autodocumentation in any style: GDOC, SPHINX, ReDOC...
 - + generation, update, check can be automated
 - + full ide and linters support
 - + collapsable, may be hidden in ide



If Type Hints Are Required:

- Type-hints required in some cases:
 - Dataclasses
 - libraries, where type hints used for validation or code generation.

Add type-hints on production from Stubs (.pyi)
or from Docstrings with **@typingdecorator**



@typingdecorator

```
@typingdecorator
```

```
class Example:
```

```
    """
```

```
    Example dataclass.
```

```
    Attributes
```

```
    -----
```

```
    foo : int
```

```
        The foo attribute. Default is 123.
```

```
    bar : str, optional
```

```
        The bar attribute.
```

```
    """
```

```
    def info(self):
```

```
        """
```

```
        :returns: str: Some info
```

```
        """
```

```
        return f"foo={self.foo}, bar={self.bar}"
```

```
# Test
```

```
>> example = Example(foo=5, bar="xyz")
```

```
>> print(example)
```

```
Example(foo=5, bar='xyz')
```

```
>> print(example.info.__annotations__  
{'return': <class 'str'>})
```

```
>> print(Example.__annotations__  
{'foo': <class 'int'>, 'bar': typing.Optional[str]})
```

MALA ♡

Type Hints recap:

- Type Hints increase cognitive and structural complexity
- Type Hints increase weight of package, and codelines count
- Stub file keep information about types, 0% refactoring in typeless projects
- doc strings can keep information about types
- `@typingdecorator` cover all cases, if type-hints are required

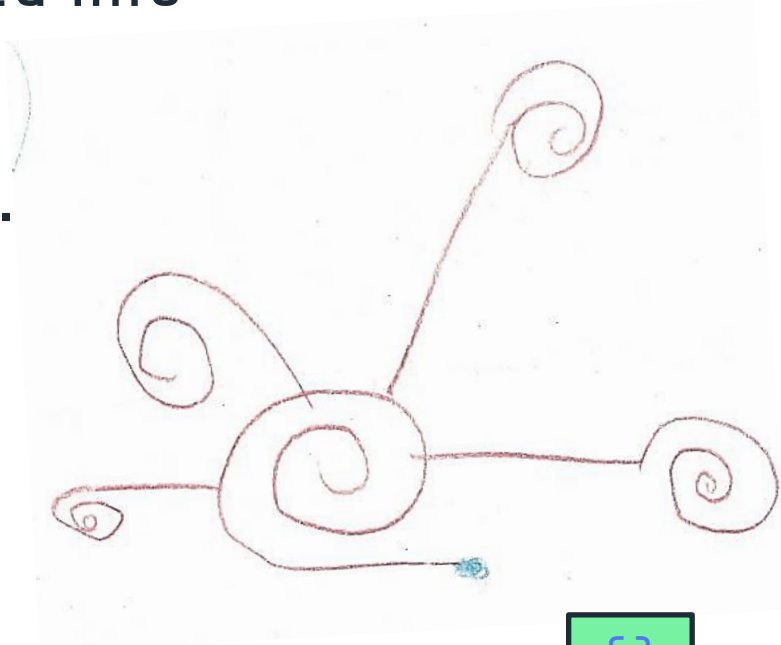


Strategies to distribute or reduce complexity

documentation

Documenting:

1. Write documentation in style „if i die, all should works“:
 - only unknown information in style „i have only last 5 seconds to tell..“
 - add security data in form „where i can get required info“
2. Allow LLM add known information, if it is important.
Or skip it.



Strategies to distribute or reduce complexity

Refactoring imports don't require testing

LLMs

Human Load vs Token

Humans

- Can operate with 7 ± 2 concepts
- Nested logic can break understanding
- Code readability is important
- Documentation should be near to code
- Attention affect on quality of work

LLMs

- Quantity of text tokens
- Quality of text tokens
- Price per token



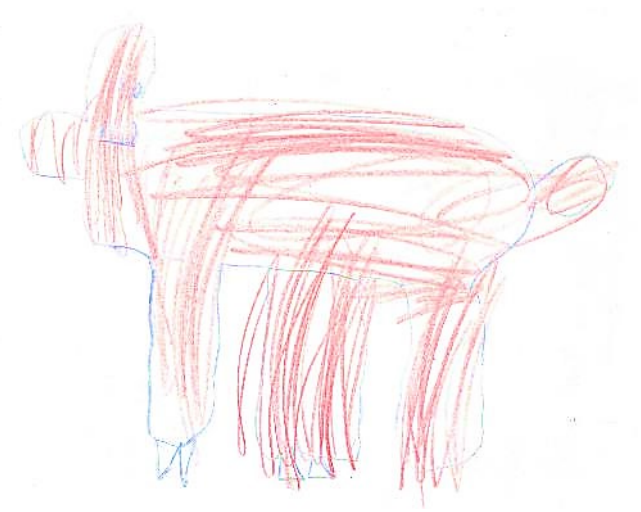
AI can and can't

Can

- Summarize statistically
- Perform reviews
- Add Testing
- Collect and Write known Documentation

Can't

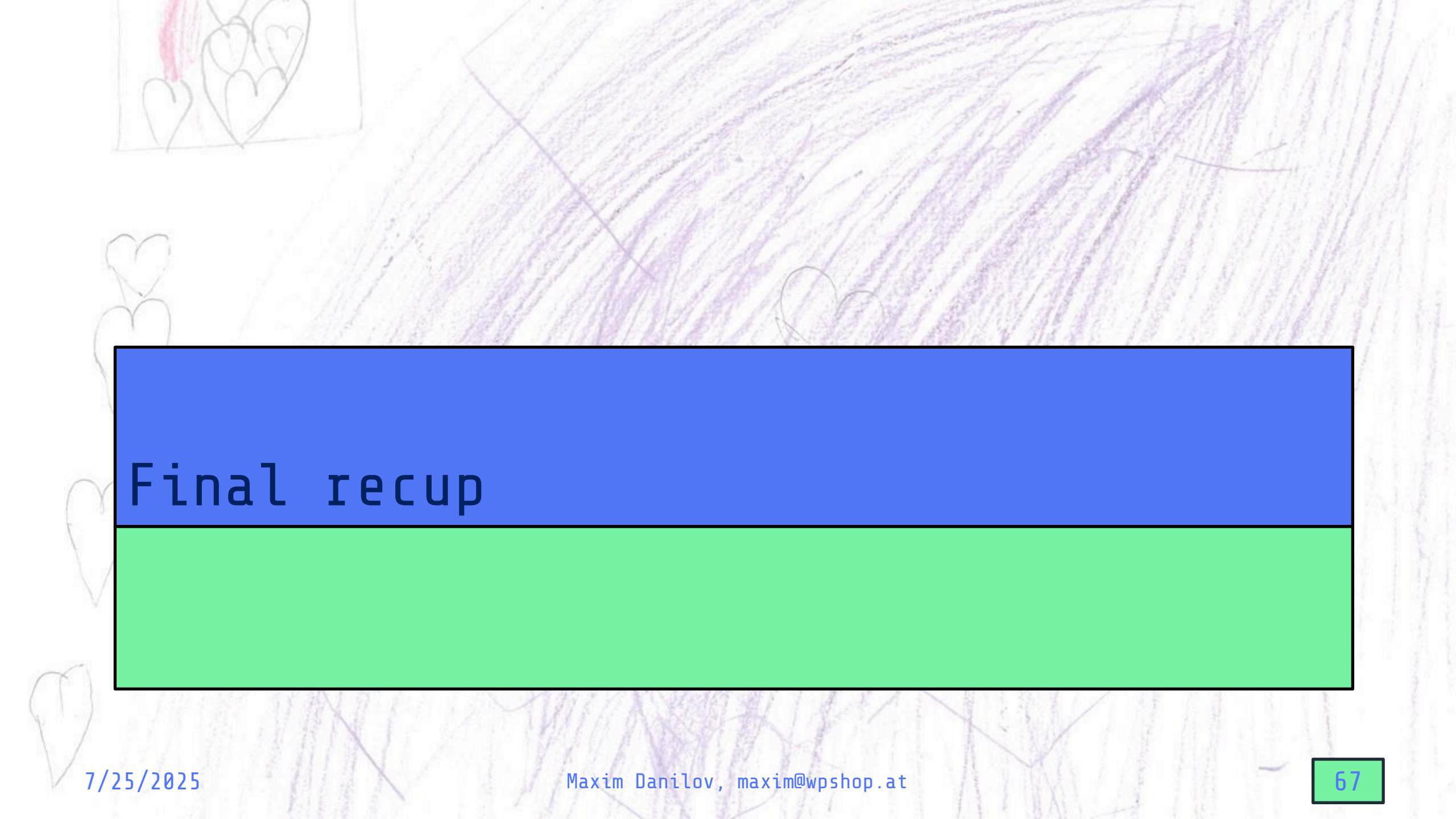
- Can't experience complexity as humans do



Cognitive Load Matters

Complexity exists only in human minds,
not in code itself.





Final recap

Summary :

1. Work with complexity lead to better design
2. Measure complexity! You can't fix what you don't measure.
3. Smart refactoring in a huge library is possible
4. We can isolate selected files and pack them into a separate package
5. Type hints increase complexity and coupling
6. Frequently imported internal dependencies: Extract. Isolate. Simplify



Try it yourself

- Take a project
- Find zero tech debt zone. Get one file from there
- perform extraction for this file
- perform measurement
- Apply some of ideas from this presentation
- perform measurement again
- Compare the results and observe the impact!



Grazie mille Bologna and you for attention!



Maxim Danilov
maxim@wpshop.at



Slides: **bitbucket.org/danilovmy/pyconit25/**

Useful links

- <https://martinfowler.com/bliki/TechnicalDebt.html>
- <https://www.agileconnection.com/article/getting-empirical-about-refactoring>
- <https://www.forbes.com/councils/forbestechcouncil/2023/09/14/taking-a-more-strategic-approach-to-the-8020-rule-in-software/>
- <https://sandimetz.com/blog/2017/9/13/breaking-up-the-behemoth>
- <https://x.com/kelseyhightower/status/1621184564956893189>

Software developing research

- **Forrester Research**
- <https://www.forrester.com/report/the-state-of-application-development-2024/RES181303>
- **McKinsey & Company**
- <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/developer-velocity-how-software-excellence-fuels-business-performance>
- **CAST**
- <https://www.castsoftware.com/news/cast-software-intelligence-report-application-age>
- **Michael Feathers**
- stickyminds.com/article/getting-empirical-about-refactoring

Easter Egg Slide

- If complexity had a face...

```
def timing[**P, R] (f: Callable[P, R]) -> Callable[P, R]:  
    def inner(*args: P.args, **kwargs: P.kwargs) -> R:  
        t = time.time()  
        retval = f(*args, **kwargs)  
        print(f"{f} took {time.time() - t} seconds")  
    return retval  
return inner
```

Easter Egg Slide

- If complexity had a face...

ule_extractor > legacy > api > infrastructure_services > aws >  aws_client_service.py

```
class AWSClientService:
    ... def __init__(self, aws_access_key_id: int, aws_secret_access_key: str, aws_region_name: str):
    ...     self._aws_access_key_id = int(aws_access_key_id)
    ...     self._aws_secret_access_key = aws_secret_access_key
    ...     self._aws_region_name = aws_region_name
    ...     self._aws_session = None
```